

Laboratory workbook

Jason Karpeles | Constructed exercises and conditional calculations

The original 23 workbench problems and the closing task are preserved below. Each chapter section then prints the inputs for its default, changed and transfer cases, so its questions can be worked without opening a notebook. The three capstone questions follow the chapter sections. All values are constructed unless a question explicitly says otherwise. Use the separate solutions after attempting a problem.

Original Mathematical Workbench

An equation becomes usable when a reader can decide what belongs in it, calculate its answer, and recognize when that answer has lost its warrant. This workbench gives each of the six parts a calculation, a diagnosis, and a problem in which an assumption changes.

All cases and numerical values below are constructed exercises, not measurements of deployed systems. Treat stipulated probabilities as known within each exercise unless the question explicitly describes them as estimates. Utilities, costs, and rewards share units only where the problem says they do. Component population scores are $U_V(S)$; a finite measured average is $\hat{U}_V(S)$. Outcome utilities remain $u(y)$, realized states remain x_t , and beliefs remain b_t . Each problem uses only symbols defined in the chapters it cites, plus the local labels it introduces.

Work through the questions before turning to the solutions.

Technical Requirements

The original eighteen problems and five expansion problems require only the book and a way to calculate; a calculator is optional. To run the optional simulator probes, use Python 3.10 or later and a terminal on Windows, macOS, or Linux. The script uses the standard library and calls no external model or service. Obtain the companion files from the book's companion materials and keep the `scripts` folder beneath the working folder. Run the commands in "Simulator probes" and "Running the expansion probes" from that working folder.

The complete laboratory package adds 29 notebooks (one for each of the 27 chapters, an orientation, and a document-release capstone) and 28 assistant skills (one for each chapter plus a master skill). Open `guide/index.html` to read the executed calculations without installing Python. Open `START-HERE.md` for the launcher, editable inputs, and skill installation. Notebook execution requires Python 3.11 or later and the listed notebook dependencies; the standard-library chapter commands also support Python 3.10. The master skill routes the implemented methods and their input contracts. Its calculations use declared local models unless the reader supplies compatible data, and their assumptions remain part of the result. The standalone workbook preserves these original problems and adds chapter exercises with separate solutions.

Part I: How New Behavior Appears

I.1 Calculation: an interaction with a declared base

Use Chapters 2 and 5. A frozen model is evaluated on the same task distribution under a fixed evaluation contract. Retrieval and a typed tool are either enabled or disabled. The resource allowance, evaluator, sampling settings, and stopping rule are identical across conditions. The entries are stipulated mean scores on a scale from zero to one, not noisy sample estimates.

Assembly	Score
$U(\{\theta\})$	0.40
$U(\{\theta, r\})$	0.52
$U(\{\theta, t\})$	0.49
$U(\{\theta, r, t\})$	0.73

Declare the base set and calculate $\Gamma(r; t)$. Decompose the joint improvement into the two isolated gains and their interaction. Check whether ρ_Γ is defined under the book's positivity conditions, then calculate it. Finally, distinguish the assertion that the combined assembly has the highest score from the assertion that retrieval and the tool cooperate positively. Which comparison establishes each assertion? Explain whether these scores establish how the cooperation occurs inside the system.

I.2 Diagnostic: a forecast that saw its answer

Use Chapter 3. A team wants to forecast the interaction contrast of a model family member before assembling its agent. It collects predictor features z_m , chooses a family of forecasting rules with parameters ψ , and reports accurate predictions of Γ_m on a bank called held out. During development, however, the team repeatedly examines that bank's realized contrasts, uses them to choose the features, and stops revising when the prediction errors look small. It then fits the selected rule on the training members alone.

Is that enough to make the reported test prospective? Identify the information path that matters and describe a repair that leaves the current work useful. Also say what would have to be recorded if the advertised forecast were made before the tool and retrieval components were available for evaluation.

I.3 Changed assumption: the joint run gets extra resources

Return to I.1. The combined assembly actually received twice the resource allowance used by the other conditions. A subsequent matched rerun restores the original allowance for every assembly. The first three scores remain unchanged, while $U(\{\theta, r, t\})$ becomes 0.58. Treat these rerun means as stipulated values under the corrected contract.

Recalculate the joint gain, $\Gamma(r; t)$, and ρ_Γ . Can the combined assembly still be the best of the four while its interaction is negative? Explain how that outcome differs from saying that the tool is harmful in every context. Calculate the tool's improvement without retrieval and its improvement with retrieval. State which original claim must be withdrawn and which matched claim can replace it. Do not interpret the difference between the two joint scores as a separately identified causal effect of resources without explaining what the construction

actually holds fixed.

Part II: Choice

II.1 Calculation: pay now to improve a later decision

Use Chapters 6 and 7. An authorized release produces reward 100 if correct and zero if incorrect. The current release is correct with probability 0.80. Alternatively, the agent can verify once, paying reward -6 immediately. Verification deterministically leads to a state where an authorized release is correct with probability 0.95. No further verification is allowed. Release ends the episode, and terminal value is zero. Both choices remain authorized throughout.

Input	Value
Correct-release reward	100
Incorrect-release reward	0
Current correctness	0.80
Verified correctness	0.95
Immediate verification reward	-6
Discount factor γ	0.90

Compute the verified state's optimal value, the current value of releasing immediately, and the current value of verification followed by release. Apply the convention that a reward on the transition out of the current state is undiscounted; discount only the next state's value. Which action should the agent take? Find the discount factor at which the two current actions tie. Explain why comparing the immediate verification cost with the immediate expected release reward is the wrong decision procedure.

II.2 Diagnostic: a return that forgets remaining opportunities

An implementation of II.1 stores only the current document in x_t . It omits the remaining verification allowance. After verification, it leaves the same action menu available and offers another verification at the same cost. Its designer says the Bellman equation will eventually decide that verification is no longer worthwhile, so the missing allowance does not matter.

Identify the state distinction required by the stipulated problem. Explain why a plausible answer from the implementation would not establish that it solved that problem. Give a corrected description of the unverified state, the verified state, their available actions, and the terminal state. State where the budget belongs in Chapter 5's composite state, and explain what additional evidence would be needed before replacing the one-verification problem with a model of repeated verification. Finally, test whether the two states could share one exact κ class: compare their feasible action sets, terminal/permission labels, rewards, and transition mass into each proposed abstract class.

II.3 Changed assumption: verification consumes the deadline

Keep every payoff, probability, and action restriction from II.1, but change γ to 0.75. Recompute the verification route and compare it with immediate release.

Explain what has changed about the optimal action and what has not changed about the evidence. Then consider a separate contractual change: the release permission expires before the verified state is reached. Under that change, may the verified release value from II.1 still be used? State which part of the model must change and what information is missing for a numerical answer. Distinguish discounting a still-authorized delayed action from removing that action from the feasible set.

Part III: Planning and Learning

III.1 Calculation: which arm earns the next pull?

Use Chapter 11's UCB rule. There are two actions with stationary rewards bounded between zero and one. Each has already been sampled. Use $c = \sqrt{2}$ and the natural logarithm.

Action	Completed pulls $N_t(a)$	Empirical mean $\hat{\mu}_a$
A	80	0.70
B	20	0.60

Calculate t , each exploration bonus, and each full index, rounding the final indices to four decimal places. Which action receives the next pull? Explain why the selected action need not have the larger empirical mean and why an index above one is not an assertion that the action's expected reward exceeds one. State the limitation of calling the bonus a value of information.

III.2 Diagnostic: information that cannot change the choice

Use Chapters 8 and 11. A hidden state is equally likely to be good or bad. Acting yields utility 12 in the good state and -2 in the bad state. Abstaining yields zero in either state. A free observation has two equally likely outcomes. After the first outcome, the posterior probability of the good state is 0.60; after the second, it is 0.40. These posteriors are consistent with the stated prior. The observation arrives before the decision and has no effect on the state or action set.

Calculate the expected utility of acting before observation and after each possible observation. Find the gross $VOI(O)$ by comparing the expected optimized utility after observation with the best utility beforehand. Does learning about the hidden state necessarily improve this decision? Identify the relevant action threshold. Explain why a system that reports the observation's information content has not yet reported its decision value, even when its report of information content is correct.

III.3 Changed assumption: the same evidence meets higher stakes

Keep the prior, observation probabilities, posterior probabilities, and abstention payoff from III.2. Change only the utility of acting in the bad state to -10. The good-state utility remains 12. An observation now costs 0.50 utility units, paid regardless of its outcome, and has no other effect.

Compute the best action before observing and after each possible observation. Find the gross value of observation, subtract its cost, and decide whether to acquire it. Identify the largest observation cost consistent with a weak preference for acquisition. Explain why the answer can change even though the information channel is identical.

Part IV: Agent Architectures

IV.1 Calculation: a lost response and an uncertain effect

Use Chapter 17's deliberately simplified retry decision. The response to an authorized non-idempotent tool call was lost. The original attempt is finished or cancelled with a guarantee against later application; the effect either landed or did not. The belief that it landed is $\beta = 0.30$. A retry certainly applies the effect if it was absent and duplicates it if it was present. There are no other failures, delay costs, or request costs in this construction. Declining leaves the world as it stands. Use negative cost as utility.

Input	Value
Belief effect landed β	0.30
Duplicate cost c_{dup}	40
Missing-effect cost c_{miss}	10

Compute the expected cost of retrying and of declining. Calculate the retry threshold and the expected-utility difference in Equation 17.3. Select the preferred action under these assumptions.

IV.2 Diagnostic: identical prompts, different worlds

Two executions have exactly the same visible prompt: the request was sent, the response timed out, and no further result is available. In one execution the service applied the effect before losing the response. In the other the request never reached the service. The operation adds a new ledger entry each time it is applied. Retrying does not replace an existing entry.

A developer hashes the prompt and treats matching hashes as proof that the two executions have the same state. Diagnose the error using $x_t = (c_t, m_t, w_t, b_t)$. Which coordinates could agree while the decision-relevant world differs? Explain the role of a belief over the possible effect states when the agent cannot inspect the service directly. Describe one observation that would distinguish these executions and one change to the tool interface that could make the distinction irrelevant to duplicate cost. Explain why known applied does not justify another non-idempotent append, while terminal non-application can permit a fresh authorized attempt. Explain why an empty ledger is insufficient if the original request is still in flight. Do not assume that a successful compensation erases the original entry or its consequences. If a repair is available, state what must be verified before it can support another attempt.

IV.3 Changed assumption: purchase a decisive read

Keep IV.1's belief and costs. Before choosing whether to retry, the agent may purchase a read that perfectly reveals whether the effect landed. The read costs 1 utility unit, creates no effect, and is guaranteed to finish while the original authorization remains valid. After the read, the agent retries only if the effect is absent. Calculate the gross value of this observation and the net gain over the best action without it.

Now consider a separate interface revision: repetition has exactly the intended effect of one application, including after the original uncertain response. In the simplified cost model this makes $c_{\text{dup}} = 0$; all other assumptions remain. Recompute the best action without the read. How much is the same perfect observation worth now, before paying for it? Explain what this comparison reveals about improving observability versus improving the effect semantics of the tool.

Part V: Societies of Agents

V.1 Calculation: charge for the delay imposed on others

Use Chapter 21's directed Braess network. Total flow is one and travelers are nonatomic. Edges S-U and L-T have latency equal to their own flow. Edges U-T and S-L have constant latency one. A directed U-L edge has latency zero. The available routes are S-U-T, S-L-T, and S-U-L-T, carrying q_{upper} , q_{lower} , and q_{middle} respectively.

At the split with half the flow on each outer route and none on the middle route, calculate every route's physical latency and the system's total physical latency. Is that split an equilibrium when each traveler minimizes physical latency? Then replace the experienced edge cost by Chapter 21's marginal-cost latency $\ell_e^{\text{mc}}(q_e)$. Recalculate the three experienced route costs at the same split. Is the split now an equilibrium? Explain why the corrected experienced cost should not be substituted for physical latency when reporting the amount of travel delay.

V.2 Diagnostic: the final confirmation disappears

Use Chapter 20's finite handshake construction. A sends a proposal to B. On receipt, B sends an acknowledgement. On receiving that acknowledgement, A sends a confirmation. A acts if it has received the acknowledgement; B acts only if it has received the confirmation. No message beyond these is sent, and the channel may lose any message. Silence reveals nothing about delivery. The no-message outcome is that neither party acts.

Compare an execution where all messages arrive with one where only the final confirmation is lost. Write each party's relevant local history and action in both executions. Which party cannot distinguish the executions? Does the rule guarantee that either both act or neither acts? Explain why adding one more acknowledgement does not, by itself, establish such a guarantee for a revised rule. Your answer should identify the observation available to a final receiver and missing from the final sender, rather than merely cite the name of an impossibility result.

V.3 Changed assumption: remove the shortcut

Return to V.1, restore physical latency as every traveler's objective, and remove the U-L edge. Derive the equilibrium outer-route split and its total latency. Compare that result with the equilibrium when the zero-latency U-L edge is available under the same physical-cost objective. Show why all flow on the middle route is an equilibrium in the latter network; also explain why any positive outer-route flow would fail equilibrium there.

Calculate the ratio of the shortcut network's equilibrium latency to the best physical latency achievable in that network. Chapter 21's expression $1.5 + 0.5q_{\text{middle}}^2$ gives the minimum total latency at a fixed middle flow; explain what is being minimized. State what this example does and does not support about adding connections to a society of agents.

Part VI: Trust

VI.1 Calculation: hard safety stays outside the score

Use Chapters 22 and 27. The following numbers are exact stipulated evaluations of two policies under a common horizon and distribution. Expected cost must not exceed 1. A proposed optimizer instead maximizes expected reward minus twice expected cost. Randomization means choosing one policy before the episode, so rewards and expected costs mix linearly.

Policy	Expected reward	Expected cost
Careful	7	0.50
Aggressive	11	2.00

Calculate each penalized score and identify the selected policy. Check feasibility under the actual expected-cost constraint. If mixing is permitted, determine the largest feasible probability of choosing Aggressive and the resulting expected reward. Then suppose Aggressive also contains an action excluded from $\mathcal{A}_{\text{auth}}(x)$ at the current state. Can the mixture's acceptable expected cost make that action available? Explain the distinction between an expected resource constraint and current authorization.

VI.2 Diagnostic: a guard result becomes development evidence

Use Chapter 25's constructed guard rule. A fixed candidate procedure has $m=200$ independently drawn paired guard cases, each difference bounded in $[-1, 1]$. The declared uplift threshold is $\tau = 0.25$, and the candidate was fixed before guard access. Compute the one-candidate false-accept bound $\exp(-(200)(0.25)^2/2)$ and the union-bound upper limit for ten separately predeclared candidates meeting the same assumptions.

Now suppose the second guard result was shown to the procedure designer, who uses it to revise the third candidate. Does the ten-candidate accounting still support a release claim for that revised candidate? State what evidence role the consulted guard has acquired and what is needed before release.

Finally, the revised candidate clears the observed uplift threshold, has $\hat{C}_G(\phi') \leq c - \varepsilon_{\text{safe}}$, but lacks authority for its proposed canary action class. Identify the acceptance condition that blocks release. State one owner, rollback trigger, deadline, and fallback that the canary record must name.

VI.3 Changed assumption: the destination may miss its deadline

Use Chapter 27. Acting now is correct with probability 0.80. A correct resolution has utility 10 and an incorrect resolution has utility -10. A designated destination is correct with probability $p_E(x)=0.95$ when it returns by deadline Δ . A timely delegation incurs a fee of 1, so under guaranteed timely return it can be compared directly with acting. Both routes are authorized.

Change the guarantee: the destination now returns on time with probability 0.60. The fee is paid whenever delegation is initiated, even if no reply arrives. A timely reply retains conditional correctness 0.95. A missed deadline triggers an authorized fallback with utility 2, and no further benefit or cost. Compute the immediate-action value, the guaranteed-return delegation value, and the changed delegation value. Which route wins before and after the change? Find the minimum timely-return probability at which delegation weakly matches acting. Explain why a destination's conditional accuracy alone cannot answer this question.

Expansion problems

The five problems below extend the existing eighteen. They use the same document-release contract: an intended object, current version and recipient, current authority, a budget, and a confirmed effect. Their numerical inputs are constructed. Solve them before inspecting the separate solutions.

E.1 Computer use: the saved coordinate

Use Chapter 18. The saved screen puts approved document v2 above unapproved v3. The current screen may preserve that layout or swap the rows. A coordinate policy uses the saved upper-row point. A semantic policy requests v2 by identifier. A transactional policy additionally supplies the saved state version.

1. Assign probability 0.6 to the unchanged layout and 0.4 to the swapped layout. Calculate intended completion and unauthorized release for the coordinate policy without mediation. Then add a monitor that denies every release of v3.
2. In the constant-rate freshness construction, use 0.02 material changes per second and delays of 5, 15, and 30 seconds. Calculate the no-invalidating-change probabilities. Identify what the rate would need to count.
3. A perfect fresh observation costs 1 utility unit. A wrong coordinate has probability 0.2. Compare a denied-attempt cost of 3 with an unguarded harmful-effect cost of 40, both in utility units. Assume observation is followed immediately by execution. Which information purchase pays?

Use the computer-use fixture to inspect which object each command reaches. A transactional denial after a state-version change is different from selecting the wrong document.

E.2 Learning: feedback that changes the target

Use Chapter 13. The direct route has success probability 0.55. Retrieval raises it to 0.80 and costs 0.05 utility units. Success has utility 1 and failure has utility 0. Let p be the retrieve probability.

1. Calculate expected external utility at p equal to 0, 0.5, and 1.
2. Add 0.30 reward whenever retrieval is avoided. Calculate the same three expected training returns. Does the training objective still prefer the route with higher external success?

3. Use rewards 0 and 1, state potentials 0, 0.4, and 0, and discount one. Calculate shaped rewards and their sum. Repeat with terminal potential 0.2. Explain what changed.

Run the learning probe with separate training and evaluation seeds. Distinguish its exact population quantities from sampled successes. Explain why a successful update under the declared reward need not improve the external task.

E.3 Orchestration: parallel work and shared error

Use Chapters 19 to 21. Two independent evidence checks each take three seconds. The supervisor's dispatch and combination consume two further seconds. Each check costs two work units; final verification costs one. Team dispatch and combination add three work units. The serial procedure has no supervisor overhead, so its time is the checks alone. The reported time covers evidence checks and supervisor overhead; final verification duration is excluded from both procedures.

1. Compare serial and parallel elapsed times and total costs when the service can run two checks at once.
2. Change capacity to one. What remains of the latency advantage?
3. A shared-source failure has probability 0.1. Otherwise, workers fail independently with probability 0.05 each. Calculate each marginal failure probability, joint failure, and the product of marginals.
4. The parent holds read and verify capabilities. The worker requests read and release. Determine its granted capability set under intersection.

A faster workflow must still satisfy the same authority contract. Do not count requested capabilities as granted ones.

E.4 Resources: a useful ratio with a missing denominator

Use Chapter 16. Procedures A, B, and C have respective per-attempt costs of 1, 1.5, and 2 monetary units; authorized-success probabilities 0.60, 0.80, and 0.85; and latencies 1, 2, and 5 seconds.

1. Calculate population cost per authorized success, including failed-attempt costs.
2. Apply a minimum success requirement of 0.75 and a three-second deadline. Identify feasible procedures.
3. Extend the deadline to ten seconds and assign each failed task a loss of 20 monetary units. Compare expected combined costs for B and C.
4. Two attempts cost 2 and 3 monetary units; neither succeeds. Report total cost, successes, and empirical cost per success.

State which quantities are population inputs and which describe the two observed attempts. A permission violation would make an otherwise attractive procedure infeasible.

E.5 Evaluation: repeat the task before averaging

Use Chapter 24. Two equally weighted tasks have success probabilities 0.2 and 0.8. Repeated runs are independent conditional on task, with identical procedure and reset rules.

1. Calculate mean single-run success, average all-success probability for two runs, and average at-least-one success probability.
2. Compare those results with squaring the mean and applying coverage to the mean. Explain which experiment each calculation describes.
3. A recorded bank has three successes and one failure. Calculate the fraction of two-run subsets with all successes and with at least one success.
4. Unfamiliar-task success is 0.4. A constructed evaluation contains 20 percent exposed tasks with success probability one. Calculate aggregate success. State what this number cannot establish about unfamiliar-task improvement.

Closing task: one request, one record

C.1 Cumulative: from drills to a decision record

Treat II.1, IV.1, and VI.3 as three stages of one constructed request, R-7, for document version v2. In stage one the agent chooses between verifying and releasing. In stage two the release times out. In stage three the agent chooses between acting again and delegating to the document owner. Stipulate that acting again means a second submission of R-7. Each case declares its own utility scale, so do not add or compare values across stages.

Complete one row per stage with these fields: belief about the effect and its source; action chosen; value, threshold, or flip condition. Then record what evidence does not establish, what remains unresolved with an owner, deadline, and fallback, which earlier problem's method does not apply to R-7, and where two stages conflict and which one binds.

Chapter 1: Did capability jump, or did the scoring rule create a cliff?

Inputs. All values are constructed. Reading page and notebook: 01-score-threshold.html. The questions below refer to these three cases.

Field meanings:

- scales: Finite strictly increasing numeric scale values.
- scores: Aligned continuous task scores in $[0,1]$.
- threshold: Pass cutoff in $[0,1]$.

Default case:

```
scales = [1, 2, 3, 4, 5]
scores = [0.42, 0.46, 0.49, 0.52, 0.56]
threshold = 0.5
```

Changed case (any field not listed keeps its default value):

```
threshold = 0.55
```

Transfer case (any field not listed keeps its default value):

```
scales = [10, 20, 40, 80]
scores = [0.1, 0.3, 0.6, 0.8]
threshold = 0.7
```

Questions:

1. List default pass flags.
2. Does changing only the cutoff prove model change?
3. What is the first transfer slope?

Chapter 2: What did these two components contribute together beyond their isolated gains?

Inputs. All values are constructed. Reading page and notebook: 02-four-cell-interaction.html. The questions below refer to these three cases.

Field meanings:

- scores: Four finite scores ordered neither,first only,second only,both.
- budgets: Four nonnegative matched resource budgets in the same order.

Default case:

```
scores = [0.2, 0.3, 0.25, 0.8]
budgets = [10, 10, 10, 10]
```

Changed case (any field not listed keeps its default value):

```
budgets = [10, 10, 10, 20]
```

Transfer case (any field not listed keeps its default value):

```
scores = [0.2, 0.5, 0.55, 0.7]
budgets = [8, 8, 8, 8]
```

Questions:

1. Verify the decomposition.
2. What changes when only the intact budget doubles?
3. Is -0.30 a cooperation fraction in the transfer case?

Chapter 3: Can a frozen forecast predict unseen capability observations?

Inputs. All values are constructed. Reading page and notebook: 03-prospective-forecast.html. The questions below refer to these three cases.

Field meanings:

- `development_x`: At least two development scales with variation.
- `development_y`: Aligned finite development observations.
- `test_x`: Separate unseen scales, with no development overlap.
- `test_y`: Aligned withheld observations used only for scoring.
- `family`: linear or log, declared before inspecting test outcomes.

Default case:

```
development_x = [1, 2, 3]
development_y = [0.2, 0.3, 0.4]
test_x = [4, 5]
test_y = [0.5, 0.6]
family = "linear"
```

Changed case (any field not listed keeps its default value):

```
test_y = [0.8, 0.95]
```

Transfer case (any field not listed keeps its default value):

```
development_x = [1, 2, 4]
development_y = [0, 0.69314718056, 1.38629436112]
test_x = [8, 16]
test_y = [2.07944154168, 2.77258872224]
family = "log"
```

Questions:

1. What are default fit coefficients?
2. Does the changed test update the fit?
3. Why must log-family scales be positive?

Chapter 4: Is there a permitted path from the current state to the intended effect?

Inputs. All values are constructed. Reading page and notebook: 04-permission-reachability.html. The questions below refer to these three cases.

Field meanings:

- `nodes`: Unique nonempty state identifiers.
- `start`: Registered initial node.
- `goal`: Registered desired node.
- `edges`: Directed edge list with `from,to:registered identifiers` and `allowed:exact boolean`.

Default case:

```

nodes = ["draft", "review", "release", "archive"]
start = "draft"
goal = "release"
edges = [{"from": "draft", "to": "review", "allowed": true}, {"from": "review", "to": "release",
"allowed": true}, {"from": "review", "to": "draft", "allowed": true}, {"from": "draft", "to":
"archive", "allowed": true}]

```

Changed case (any field not listed keeps its default value):

```

edges = [{"from": "draft", "to": "review", "allowed": true}, {"from": "review", "to": "release",
"allowed": false}, {"from": "review", "to": "draft", "allowed": true}, {"from": "draft", "to":
"archive", "allowed": true}]

```

Transfer case (any field not listed keeps its default value):

```

nodes = ["queued", "approved", "sent"]
start = "queued"
goal = "sent"
edges = [{"from": "queued", "to": "approved", "allowed": false}, {"from": "approved", "to": "sent",
"allowed": true}]

```

Questions:

1. What is the default shortest authorized path?
2. Does the changed raw graph still reach release?
3. Why is the transfer send action unusable?

Chapter 5: What system does a frozen chooser become when its surrounding kernel changes?

Inputs. All values are constructed. Reading page and notebook: 05-composite-kernel.html. The questions below refer to these three cases.

Field meanings:

- `step_success`: Marginal equal-step probability in $[0,1]$ for separate constructed reliability cases.
- `steps`: Positive integer transition count up to 1000.
- `conditional_success`: List of probabilities conditioned on all previous steps succeeding, one per step; its length must equal steps.
- `chooser`: State by action row-stochastic matrix.
- `tool`: Action by next-state row-stochastic matrix.
- `initial`: Normalized state distribution matching the composed kernel.

Default case:

```

step_success = 0.99
steps = 100
conditional_success = 0.99 repeated 100 times
chooser = [[0.8, 0.2], [0.3, 0.7]]
tool = [[0.9, 0.1], [0.2, 0.8]]

```

```
initial = [1, 0]
```

Changed case (any field not listed keeps its default value):

```
tool = [[0.6, 0.4], [0.1, 0.9]]
```

Transfer case (any field not listed keeps its default value):

```
step_success = 0.9
steps = 3
conditional_success = [0.9, 0.8, 0.7]
chooser = [[1, 0], [0, 1]]
tool = [[0.7, 0.3], [0.4, 0.6]]
initial = [0.5, 0.5]
```

Not printed: assemblies (five assembly variants used only by the notebook figure; see the guide page).

Questions:

1. Compute the default first kernel row.
2. Why can an independent-step model and a shared good-or-bad-condition model both have step marginals p ?
3. Compute the transfer conditional product.

Chapter 6: Should this controller release, request review, or abstain?

Inputs. All values are constructed. Reading page and notebook: 06-expected-utility.html. The questions below refer to these three cases.

Field meanings:

- **actions:** Nonempty list of uniquely named alternatives. Each has **name:**string, **probabilities:**normalized numeric list, **utilities:**aligned finite numeric list, **cost:**nonnegative number. Include abstention explicitly.

Default case:

```
actions = [{"name": "release", "probabilities": [0.9, 0.1], "utilities": [10, -50], "cost": 0},
{"name": "review", "probabilities": [1], "utilities": [2], "cost": 0}, {"name": "abstain",
"probabilities": [1], "utilities": [0], "cost": 0}]
```

Changed case (any field not listed keeps its default value):

```
actions = [{"name": "release", "probabilities": [0.9, 0.1], "utilities": [10, -100], "cost": 0},
{"name": "review", "probabilities": [1], "utilities": [2], "cost": 0}, {"name": "abstain",
"probabilities": [1], "utilities": [0], "cost": 0}]
```

Transfer case (any field not listed keeps its default value):

```
actions = [{"name": "retry", "probabilities": [0.7, 0.3], "utilities": [8, -12], "cost": 1},
{"name": "escalate", "probabilities": [1], "utilities": [1], "cost": 0}]
```

Questions:

1. Calculate release utility in the default case.

2. What first-outcome probability ties review under the default utilities?
3. Why does the transfer case not have a unique utility winner?

Chapter 7: When should a controller accept an immediate payoff instead of preparing a better future?

Inputs. All values are constructed. Reading page and notebook: 07-finite-horizon-planning.html. The questions below refer to these three cases.

Field meanings:

- horizon: Integer decision count 1..200.
- discount: Gamma in $[0,1]$.
- start: State key.
- states: Object mapping states to optional terminal:number and actions:list. Action has name,reward:number,probabilities:normalized list,next_states:aligned registered state keys.

Default case:

```
horizon = 3
discount = 1
start = "draft"
states = {"draft": {"actions": [{"name": "cash", "reward": 2, "probabilities": [1], "next_states": ["done"]}, {"name": "prepare", "reward": -1, "probabilities": [1], "next_states": ["ready"]}}, {"ready": {"actions": [{"name": "release", "reward": 6, "probabilities": [1], "next_states": ["done"]}}, {"done": {"actions": [{"name": "stop", "reward": 0, "probabilities": [1], "next_states": ["done"]}]}}
```

Changed case (any field not listed keeps its default value):

```
horizon = 1
```

Transfer case (any field not listed keeps its default value):

```
horizon = 2
discount = 0.5
start = "wait"
states = {"wait": {"actions": [{"name": "now", "reward": 1, "probabilities": [1], "next_states": ["end"]}, {"name": "later", "reward": 0, "probabilities": [1], "next_states": ["paid"]}}, {"paid": {"actions": [{"name": "collect", "reward": 4, "probabilities": [1], "next_states": ["end"]}]}, {"end": {"actions": [{"name": "stop", "reward": 0, "probabilities": [1], "next_states": ["end"]}]}}
```

Questions:

1. Compute draft value with two decisions.
2. Why does horizon 3 not exceed horizon 2 here?
3. What is transfer later value?

Chapter 8: Should a controller act now or buy an observation before choosing?

Inputs. All values are constructed. Reading page and notebook: 08-belief-information.html. The questions below refer to these three cases.

Field meanings:

- **belief:** Normalized prior state probability list.
- **transition:** Square row-stochastic state transition matrix.
- **observation:** State by observation row-stochastic matrix.
- **observed:** Zero-based observation index.
- **action_rewards:** Action by state finite reward matrix.
- **observation_cost:** Nonnegative cost paid before the one later action.

Default case:

```
belief = [0.5, 0.5]
transition = [[1, 0], [0, 1]]
observation = [[0.9, 0.1], [0.1, 0.9]]
observed = 0
action_rewards = [[10, -10], [-10, 10]]
observation_cost = 1
```

Changed case (any field not listed keeps its default value):

```
observation = [[0.5, 0.5], [0.5, 0.5]]
```

Transfer case (any field not listed keeps its default value):

```
belief = [0.8, 0.2]
transition = [[0.7, 0.3], [0.2, 0.8]]
observation = [[0.8, 0.2], [0.3, 0.7]]
observed = 1
action_rewards = [[5, -4], [0, 2]]
observation_cost = 0.2
```

Questions:

1. Compute default posterior.
2. What observation cost makes default information break even?
3. Compute transfer posterior after observation 1.

Chapter 9: Does this heuristic help search without hiding a cheaper route?

Inputs. All values are constructed. Reading page and notebook: 09-astar-audit.html. The questions below refer to these three cases.

Field meanings:

- nodes: Unique graph identifiers.
- start: Registered search start.
- goal: Registered goal.
- edges: Directed from/to identifiers with finite nonnegative cost.
- heuristic: Object mapping each node to a nonnegative estimated remaining cost.
- heuristic_cost: Nonnegative declared cost of one heuristic evaluation, separate from path cost.

Default case:

```
nodes = ["S", "A", "B", "G"]
start = "S"
goal = "G"
edges = [{"from": "S", "to": "A", "cost": 1}, {"from": "A", "to": "G", "cost": 4}, {"from": "S",
"to": "B", "cost": 2}, {"from": "B", "to": "G", "cost": 1}]
heuristic = {"S": 3, "A": 4, "B": 1, "G": 0}
heuristic_cost = 0.1
```

Changed case (any field not listed keeps its default value):

```
heuristic = {"S": 3, "A": 4, "B": 10, "G": 0}
```

Transfer case (any field not listed keeps its default value):

```
nodes = ["source", "middle", "target"]
start = "source"
goal = "target"
edges = [{"from": "source", "to": "middle", "cost": 2}, {"from": "middle", "to": "target", "cost":
2}, {"from": "source", "to": "target", "cost": 7}]
heuristic = {"source": 0, "middle": 0, "target": 0}
heuristic_cost = 0
```

Questions:

1. What is the default optimal path cost?
2. Which changed heuristic condition fails?
3. What does $h=0$ produce?

Chapter 10: Does a reusable skill finish before interruption or the deadline?

Inputs. All values are constructed. Reading page and notebook: 10-option-duration.html. The questions below refer to these three cases.

Field meanings:

- discount: Gamma in $[0,1]$.
- deadline: Nonnegative integer primitive-step deadline.

- `start_time`: Nonnegative elapsed time, same unit as deadline.
- `interrupt_after`: Optional nonnegative integer executed-step cap for each option.
- `options`: Each option has `name`, `initiation: boolean`, `rewards: nonempty primitive reward list`, `terminated: boolean`, `continuation_value: number`.

Default case:

```
discount = 0.9
deadline = 5
options = [{"name": "review-release", "initiation": true, "rewards": [-1, -1, 8], "terminated": true, "continuation_value": 2}, {"name": "archive", "initiation": true, "rewards": [1], "terminated": true, "continuation_value": 0}]
```

Changed case (any field not listed keeps its default value):

```
interrupt_after = 2
```

Transfer case (any field not listed keeps its default value):

```
discount = 0.5
deadline = 2
start_time = 0
options = [{"name": "inspect", "initiation": true, "rewards": [0, 4], "terminated": true, "continuation_value": 8}, {"name": "disabled", "initiation": false, "rewards": [9], "terminated": true, "continuation_value": 0}]
```

Questions:

1. What is default continuation discount?
2. What is interrupted review-release value?
3. Why is transfer continuation contribution 2 rather than 4?

Chapter 11: How much does a finite exploration policy pay to learn which action is better?

Inputs. All values are constructed. Reading page and notebook: 11-bounded-exploration.html. The questions below refer to these three cases.

Field meanings:

- `means`: Arm success probabilities in $[0,1]$, known only to this constructed simulator.
- `rounds`: Integer pull budget 1..10000.
- `seed`: Nonnegative integer random seed.
- `pull_cost`: Nonnegative common cost per pull.

Default case:

```
means = [0.4, 0.7]
rounds = 120
seed = 7
```

```
pull_cost = 0.05
```

Changed case (any field not listed keeps its default value):

```
means = [0.7, 0.4]
```

Transfer case (any field not listed keeps its default value):

```
means = [0.2, 0.5, 0.8]
rounds = 90
seed = 19
pull_cost = 0.1
```

Questions:

1. Express default regret through counts.
2. Does common pull cost change arm pseudo-regret?
3. Express transfer regret.

Chapter 12: How should a late consequence change values assigned to earlier states?

Inputs. All values are constructed. Reading page and notebook: 12-trajectory-credit.html. The questions below refer to these three cases.

Field meanings:

- **states:** Trajectory state identifiers, one more than reward count.
- **rewards:** Finite observed immediate rewards.
- **discount:** Gamma in $[0,1]$.
- **learning_rate:** Step size in $[0,1]$.
- **lambda:** Trace decay parameter in $[0,1]$.
- **terminal:** Exact boolean distinguishing true termination from truncation.
- **values:** Initial finite value for every trajectory state.

Default case:

```
states = ["draft", "review", "done"]
rewards = [0, 1]
discount = 1
learning_rate = 0.5
lambda = 0.8
terminal = true
values = {"draft": 0, "review": 0, "done": 0}
```

Changed case (any field not listed keeps its default value):

```
terminal = false
values = {"draft": 0, "review": 0, "done": 2}
```

Transfer case (any field not listed keeps its default value):

```
states = ["a", "b", "c", "d"]
rewards = [1, 0, 2]
discount = 0.9
learning_rate = 0.2
lambda = 0
values = {"a": 0, "b": 1, "c": 0, "d": 0}
```

Questions:

1. Compute default trace update to draft.
2. What are changed return targets?
3. What should $\lambda = 0$ match?

Chapter 13: Does optimizing the verifier reward improve the external task?

Inputs. All values are constructed. Reading page and notebook: 13-finite-policy-learning.html. The questions below refer to these three cases.

Field meanings:

- rewards: Finite verifier reward per one-step action.
- success_probabilities: External success chance per action in $[0,1]$.
- terminal_potentials: Potential value of each terminal action outcome; zero is the episodic invariant condition.
- episodes: Bounded training sample count 1..10000.
- seeds: Nonempty integer seed list.
- learning_rate: Logit step size in $[0,1]$.
- discount: Gamma in $[0,1]$.
- evaluation_runs: Positive separate evaluation count.

Default case:

```
rewards = [1, 2]
success_probabilities = [0.9, 0.2]
terminal_potentials = [0, 0]
episodes = 120
seeds = [3, 11]
learning_rate = 0.08
discount = 1
evaluation_runs = 200
```

Changed case (any field not listed keeps its default value):

```
rewards = [2, 1]
```

Transfer case (any field not listed keeps its default value):

```
rewards = [0, 1]
success_probabilities = [0, 1]
terminal_potentials = [2, 0]
episodes = 100
seeds = [7, 17]
learning_rate = 0.1
evaluation_runs = 100
```

Questions:

1. Compute initial default external success.
2. Which action has the highest default verifier reward?
3. What are transfer shaped rewards?

Chapter 14: How much value error can this approximate world model introduce?

Inputs. All values are constructed. Reading page and notebook: 14-transition-model-bound.html. The questions below refer to these three cases.

Field meanings:

- `true_transition`: Square row-stochastic true kernel after fixing a policy.
- `model_transition`: Same-size approximate row-stochastic kernel for that policy.
- `rewards`: Common immediate state reward vector, finite and nonnegative.
- `discount`: Gamma in $[0, 0.9999]$.
- `horizon`: Integer reward horizon 1..1000 with zero terminal continuation.

Default case:

```
true_transition = [[0.9, 0.1], [0.2, 0.8]]
model_transition = [[0.88, 0.12], [0.22, 0.78]]
rewards = [0, 1]
discount = 0.9
horizon = 5
```

Changed case (any field not listed keeps its default value):

```
horizon = 20
```

Transfer case (any field not listed keeps its default value):

```
true_transition = [[1, 0], [0, 1]]
model_transition = [[1, 0], [0, 1]]
rewards = [1, 2]
discount = 0.5
horizon = 4
```

Questions:

1. Compute default epsilon.
2. Compute horizon 20 finite bound.
3. What if the reward model also differs?

Chapter 15: Which memories deserve a limited retrieval budget, and does compression preserve the decision?

Inputs. All values are constructed. Reading page and notebook: 15-memory-budget.html. The questions below refer to these three cases.

Field meanings:

- budget: Nonnegative integer token budget up to 10000.
- now: Nonnegative current time.
- current_version: Current document or state version identifier.
- records: At most 20 records. Each has id,tokens:positive integer,decision_value:nonnegative number,timestamp,ttl:nonnegative numbers,authority:boolean,version:string.
- original_action_values: Finite values for each original decision alternative.
- compressed_action_values: Aligned values after compression.

Default case:

```
budget = 6
now = 10
current_version = "v2"
records = [{"id": "review-v2", "tokens": 4, "decision_value": 8, "timestamp": 9, "ttl": 3,
"authority": true, "version": "v2"}, {"id": "summary", "tokens": 2, "decision_value": 3,
"timestamp": 8, "ttl": 5, "authority": false, "version": "v1"}, {"id": "old-review", "tokens": 1,
"decision_value": 100, "timestamp": 9, "ttl": 3, "authority": true, "version": "v1"}]
original_action_values = [4, 6]
compressed_action_values = [3, 5]
```

Changed case (any field not listed keeps its default value):

```
budget = 2
compressed_action_values = [6, 5]
```

Transfer case (any field not listed keeps its default value):

```
budget = 3
now = 20
current_version = "doc-C"
records = [{"id": "expired", "tokens": 1, "decision_value": 10, "timestamp": 10, "ttl": 2,
"authority": false, "version": "doc-C"}, {"id": "current", "tokens": 3, "decision_value": 4,
"timestamp": 19, "ttl": 3, "authority": true, "version": "doc-C"}]
original_action_values = [1, 0]
compressed_action_values = [2, 1]
```

Questions:

1. Why exclude old-review?
2. What is default retrieval value?
3. Does action preservation prove all future information survives?

Chapter 16: How many candidates should the controller generate under cost and deadline constraints?

Inputs. All values are constructed. Reading page and notebook: 16-sample-allocation.html. The questions below refer to these three cases.

Field meanings:

- `candidate_success`: Marginal correctness probability in $[0,1]$.
- `selector_accuracy`: Conditional probability of choosing a correct candidate when one exists.
- `sample_cost`: Nonnegative cost per generated candidate.
- `sample_latency`: Nonnegative serial time per candidate.
- `selector_cost`: Nonnegative selector expense for $n > 1$.
- `selector_latency`: Nonnegative selector time for $n > 1$.
- `deadline`: Nonnegative total time limit.
- `budget`: Nonnegative total expense limit.
- `shared_error`: Exact boolean choosing shared versus independent correctness construction.
- `sample_counts`: Nonempty list of positive integer candidate counts up to 1000.

Default case:

```
candidate_success = 0.4
selector_accuracy = 0.9
sample_cost = 1
sample_latency = 1
selector_cost = 1
selector_latency = 1
deadline = 6
budget = 6
shared_error = false
sample_counts = [1, 2, 3, 5]
```

Changed case (any field not listed keeps its default value):

```
shared_error = true
```

Transfer case (any field not listed keeps its default value):

```
candidate_success = 0.7
selector_accuracy = 0.5
sample_cost = 2
selector_cost = 3
```

```
selector_latency = 2
deadline = 3
sample_counts = [1, 2, 4]
```

Questions:

1. Compute default $n=5$ coverage and selection.
2. Why prefer n_1 under shared error?
3. What if no allocation is feasible?

Chapter 17: Did the tool act, and should an unresolved request be verified or retried?

Inputs. All values are constructed. Reading page and notebook: 17-effect-and-retry.html. The questions below refer to these three cases.

Field meanings:

- **idempotent:** Exact boolean. True declares server-side key/payload deduplication.
- **events:** Ordered event list: request has kind,key,payload,effect:boolean,ack:boolean; verify has observed_effect:boolean. All requests must concern one logical payload; distinct operations require separate traces.
- **verify_cost:** Nonnegative one-step cost of perfect verification.
- **retry_cost:** Nonnegative one-step request cost.
- **effect_probability:** Probability $[0,1]$ that the unresolved original request already took effect.
- **duplicate_cost:** Nonnegative extra harm from a duplicate effect.

Default case:

```
idempotent = false
events = [{"kind": "request", "key": "release-1", "payload": "edition-A", "effect": true, "ack": false}, {"kind": "request", "key": "release-1", "payload": "edition-A", "effect": true, "ack": true}]
verify_cost = 1
retry_cost = 0.2
effect_probability = 0.8
duplicate_cost = 10
```

Changed case (any field not listed keeps its default value):

```
idempotent = true
```

Transfer case (any field not listed keeps its default value):

```
events = [{"kind": "request", "key": "send-2", "payload": "packet-B", "effect": true, "ack": false}, {"kind": "verify", "observed_effect": true}]
verify_cost = 0.5
retry_cost = 0.1
effect_probability = 0.6
duplicate_cost = 4
```

Questions:

1. How many effects occur in the default trace?
2. What duplicate-harm value makes verification and retry tie?
3. What happens when a stored idempotency key is reused with a different payload?

Chapter 18: Does an observed interface still support an authorized action at the intended target?

Inputs. All values are constructed. Reading page and notebook: 18-interface-freshness.html. The questions below refer to these three cases.

Field meanings:

- `observation_age`: Nonnegative elapsed time since observation.
- `max_age`: Nonnegative freshness limit in the same time unit.
- `observed_version`: Version seen in the observation.
- `current_version`: Version at action time.
- `current_permission`: Exact boolean current authorization.
- `effect_confirmed`: Exact boolean receipt of desired effect confirmation.
- `coordinate_target`: Current object located at previously observed coordinates.
- `semantic_target`: Current object resolved by semantic interface.
- `wanted_target`: Intended object identifier.
- `change_rate`: Optional nonnegative rate of invalidating interface changes per unit time. When given, $\text{Fresh}(\text{delay}) = \exp(-\text{rate} \cdot \text{delay})$ from Equation 18.3 is reported.
- `delays`: Optional list of nonnegative delays in the same time unit; requires `change_rate`.

Default case:

```
observation_age = 1
max_age = 2
observed_version = "layout-1"
current_version = "layout-2"
current_permission = true
effect_confirmed = true
coordinate_target = "delete"
semantic_target = "release"
wanted_target = "release"
change_rate = 0.02
delays = [5, 15, 30]
```

Changed case (any field not listed keeps its default value):

```
observation_age = 3
```

Transfer case (any field not listed keeps its default value):

```
observation_age = 0
max_age = 1
observed_version = "page-C"
current_version = "page-C"
current_permission = false
effect_confirmed = false
coordinate_target = "submit"
semantic_target = "submit"
wanted_target = "submit"
change_rate = 0.05
delays = [2, 10]
```

Questions:

1. Why does default coordinate completion fail?
2. Can matching versions replace current permission?
3. What does observation age 3 do under $\text{max_age} = 2$?
4. With change_rate 0.02 per second, compute the no-invalidating-change probability for delays of 5, 15 and 30 seconds.

Chapter 19: Does successful self-play transfer to the partners the controller will actually meet?

Inputs. All values are constructed. Reading page and notebook: 19-cross-play-transfer.html. The questions below refer to these three cases.

Field meanings:

- **matrix:** Square policy by partner success-probability matrix, entries in $[0,1]$.
- **partner_weights:** Normalized target partner distribution.
- **supervisor_weights:** Normalized changed partner or supervisor distribution over the same columns.

Default case:

```
matrix = [[0.95, 0.2], [0.4, 0.9]]
partner_weights = [0.5, 0.5]
supervisor_weights = [0.9, 0.1]
```

Changed case (any field not listed keeps its default value):

```
partner_weights = [0.9, 0.1]
```

Transfer case (any field not listed keeps its default value):

```
matrix = [[1, 0, 0], [0.6, 0.6, 0.6], [0, 0, 1]]
partner_weights = [0.2, 0.6, 0.2]
supervisor_weights = [0.5, 0, 0.5]
```

Questions:

1. Compute default policy 1 mixture value.
2. Why does the changed mixture favor policy 0?
3. Which transfer policy wins?

Chapter 20: What can each party know after a request and an acknowledgement can be dropped?

Inputs. All values are constructed. Reading page and notebook: 20-bounded-message-knowledge.html. The questions below refer to these three cases.

Field meanings:

- `rounds`: Integer request/acknowledgement rounds 1..6; enumeration has $2^{(2 \text{ rounds})}$ worlds.
- `drop_probability`: Independent per-message drop probability in $[0,1]$.

Default case:

```
rounds = 2
drop_probability = 0.3
```

Changed case (any field not listed keeps its default value):

```
drop_probability = 0
```

Transfer case (any field not listed keeps its default value):

```
rounds = 3
drop_probability = 0.5
```

Questions:

1. Compute default disagreement.
2. Why can Bob know acknowledgement receipt at zero drops?
3. Compute transfer agreement.

Chapter 21: Can a faster-looking route make the whole workflow slower?

Inputs. All values are constructed. Reading page and notebook: 21-congestion-incentives.html. The questions below refer to these three cases.

Field meanings:

- `demand`: Positive continuous traveler or job flow in normalized units.
- `capacity`: Positive flow scale of each linear congestible edge.
- `constant_time`: Nonnegative fixed delay of each outer constant edge.

- `shortcut_overhead`: Nonnegative real delay on the shortcut.
- `shortcut_toll`: Nonnegative private incentive charge, excluded from travel-time social cost.

Default case:

```
demand = 1
capacity = 1
constant_time = 1
shortcut_overhead = 0
shortcut_toll = 0
```

Changed case (any field not listed keeps its default value):

```
shortcut_toll = 0.5
```

Transfer case (any field not listed keeps its default value):

```
demand = 0.5
```

Questions:

1. Compute default price of anarchy.
2. Why exclude toll revenue from social travel time?
3. Compute transfer social optimum.

Chapter 22: Which choices satisfy both current authority and a declared expected-risk limit?

Inputs. All values are constructed. Reading page and notebook: 22-risk-authority-contract.html. The questions below refer to these three cases.

Field meanings:

- `risk_limit`: Maximum allowed adverse-event probability in $[0,1]$.
- `risk_penalty`: Nonnegative utility deduction per unit probability.
- `actions`: Each row has name,reward:finite number,risk:probability,authorized:exact boolean. Abstention must be an explicit row.

Default case:

```
risk_limit = 0.1
risk_penalty = 5
actions = [{"name": "fast-release", "reward": 10, "risk": 0.3, "authorized": false}, {"name": "reviewed-release", "reward": 5, "risk": 0.05, "authorized": true}, {"name": "risky-authorized", "reward": 8, "risk": 0.2, "authorized": true}, {"name": "abstain", "reward": 0, "risk": 0, "authorized": true}]
```

Changed case (any field not listed keeps its default value):

```
risk_limit = 0.25
```

Transfer case (any field not listed keeps its default value):

```

risk_limit = 0
risk_penalty = 100
actions = [{"name": "act", "reward": 20, "risk": 0.01, "authorized": true}, {"name": "wait",
"reward": -1, "risk": 0, "authorized": true}]

```

Questions:

1. Which default action wins the hard constraint?
2. Can $\text{risk_limit} = 0.25$ authorize fast-release?
3. Why does transfer choose wait despite negative reward?

Chapter 23: Did untrusted data cross into control or cause an unauthorized effect?

Inputs. All values are constructed. Reading page and notebook: 23-local-security-monitor.html. The questions below refer to these three cases.

Field meanings:

- **capabilities:** List of permitted action names in the local contract.
- **current_version:** Current protected document version.
- **events:** Ordered data/review/action events. data:instruction_attempt,promoted_to_control booleans; review:valid boolean,version; action:action,authority_source=user/system/untrusted-data,version for publish,executed boolean.

Default case:

```

capabilities = ["read", "publish"]
current_version = "v2"
events = [{"kind": "data", "instruction_attempt": true, "promoted_to_control": false}, {"kind":
"action", "action": "publish", "authority_source": "untrusted-data", "version": "v2", "executed":
false}, {"kind": "review", "valid": true, "version": "v2"}, {"kind": "action", "action": "publish",
"authority_source": "user", "version": "v2", "executed": true}]

```

Changed case (any field not listed keeps its default value):

```

events = [{"kind": "data", "instruction_attempt": true, "promoted_to_control": true}, {"kind":
"action", "action": "publish", "authority_source": "untrusted-data", "version": "v2", "executed":
true}, {"kind": "review", "valid": true, "version": "v2"}, {"kind": "action", "action": "publish",
"authority_source": "user", "version": "v2", "executed": true}]

```

Transfer case (any field not listed keeps its default value):

```

current_version = "B"
events = [{"kind": "review", "valid": true, "version": "A"}, {"kind": "action", "action": "publish",
"authority_source": "system", "version": "B", "executed": false}]

```

Questions:

1. How many changed security violations occur?
2. Why does the changed plot rise only once?

3. Does a valid review of version A authorize publishing version B?

Chapter 24: What improvement is supported by these procedure-level task outcomes?

Inputs. All values are constructed. Reading page and notebook: 24-matched-capability.html. The questions below refer to these three cases.

Field meanings:

- **baseline:** Observed baseline procedure identifier.
- **candidate:** Observed distinct candidate procedure identifier.
- **records:** List of records with `procedure,task,run:string` and `success:exact boolean`; optional `cost:nonnegative number`, `exposed:exact boolean`. Procedure/task/run must be unique.
- **task_weights:** Optional object of task identifiers to normalized probabilities defining a target mixture.

Default case:

```
baseline = "base"
candidate = "new"
records = [{"procedure": "base", "task": "easy", "run": "0", "success": true, "cost": 1, "exposed": false}, {"procedure": "base", "task": "hard", "run": "1", "success": false, "cost": 1, "exposed": true}, {"procedure": "base", "task": "easy", "run": "2", "success": true, "cost": 1, "exposed": false}, {"procedure": "base", "task": "hard", "run": "3", "success": false, "cost": 1, "exposed": true}, {"procedure": "new", "task": "easy", "run": "0", "success": true, "cost": 2, "exposed": false}, {"procedure": "new", "task": "hard", "run": "1", "success": true, "cost": 2, "exposed": true}, {"procedure": "new", "task": "easy", "run": "2", "success": true, "cost": 2, "exposed": false}, {"procedure": "new", "task": "hard", "run": "3", "success": false, "cost": 2, "exposed": true}]
task_weights = {"easy": 0.5, "hard": 0.5}
```

Changed case (any field not listed keeps its default value):

```
task_weights = {"easy": 0.1, "hard": 0.9}
```

Transfer case (any field not listed keeps its default value):

```
baseline = "old"
candidate = "proposal"
records = [{"procedure": "old", "task": "alpha", "run": "one", "success": true, "cost": 1}, {"procedure": "proposal", "task": "beta", "run": "two", "success": false, "cost": 3}]
task_weights = {"alpha": 0.5, "beta": 0.5}
```

Questions:

1. Compute the default paired difference.
2. Compute candidate success under the changed hard-task weight.
3. Why is the transfer matched difference unavailable?

Chapter 25: Should the development winner be accepted under a separate release guard?

Inputs. All values are constructed. Reading page and notebook: 25-development-guard-gate.html. The questions below refer to these three cases.

Field meanings:

- `baseline_guard`: Nonempty binary baseline outcomes aligned with each candidate guard.
- `minimum_guard_gain`: Finite prespecified candidate-minus-baseline mean threshold.
- `guard_reused`: Exact boolean stating whether guard evidence influenced improvement or prior selection.
- `candidates`: Nonempty rows with `name`, `development`: binary list, `guard`: binary list matched to `baseline_guard`.

Default case:

```
baseline_guard = [true, false, true, false]
minimum_guard_gain = 0.2
guard_reused = false
candidates = [{"name": "flashy", "development": [true, true, true, true], "guard": [true, false, true, false]}, {"name": "steady", "development": [true, true, true, false], "guard": [true, true, true, false]}]
```

Changed case (any field not listed keeps its default value):

```
candidates = [{"name": "flashy", "development": [true, true, true, true], "guard": [true, true, true, false]}, {"name": "steady", "development": [true, true, true, false], "guard": [true, true, true, false]}]
```

Transfer case (any field not listed keeps its default value):

```
baseline_guard = [false, true]
minimum_guard_gain = 0.1
guard_reused = true
candidates = [{"name": "proposal", "development": [true, true], "guard": [true, true]}]
```

Questions:

1. Why not release steady by default?
2. What changed gain allows flashy to pass?
3. Does contamination erase observed gain?

Chapter 26: How much capability exists in this finite candidate bank, and how much can selection deploy?

Inputs. All values are constructed. Reading page and notebook: 26-conditional-bank-ceiling.html. The questions below refer to these three cases.

Field meanings:

- `success_matrix`: Candidate by task matrix of exact boolean task success.

- `task_weights`: Normalized task-distribution weights.
- `selected_candidates`: Zero-based candidate index chosen for each task.
- `deployment_allowed`: Exact boolean deployment permission for each task.
- `independent_candidate_success`: Probability for a separate constructed independent-candidate saturation curve.

Default case:

```
success_matrix = [[true, false, false], [false, true, false], [false, false, true]]
task_weights = [0.2, 0.3, 0.5]
selected_candidates = [0, 0, 0]
deployment_allowed = [true, true, true]
independent_candidate_success = 0.4
```

Changed case (any field not listed keeps its default value):

```
selected_candidates = [0, 1, 2]
deployment_allowed = [true, true, false]
```

Transfer case (any field not listed keeps its default value):

```
success_matrix = [[true, true], [true, false]]
task_weights = [0.5, 0.5]
selected_candidates = [1, 1]
deployment_allowed = [true, false]
independent_candidate_success = 0.6
```

Questions:

1. What are default oracle,actual,deployed values?
2. Why is changed deployment 0.5 despite perfect selection?
3. Can adding a candidate reduce oracle prefix coverage?

Chapter 27: Can human review supply missing authority within the workflow's capacity and timing contract?

Inputs. All values are constructed. Reading page and notebook: 27-delegation-queue.html. The questions below refer to these three cases.

Field meanings:

- `arrival_rate`: Nonnegative incoming task rate per declared time unit.
- `service_rate`: Positive reviewer service rate in the same unit.
- `delegation_fraction`: Fraction in $[0,1]$ entering review.
- `agent_risk_limit`: Probability threshold in $[0,1]$ for autonomous task handling.
- `tasks`: Rows with name,agent_authorized:boolean,risk:probability,deadline:nonnegative time,human_authorized:boolean,review_received:boolean.

Default case:

```
arrival_rate = 4
service_rate = 3
delegation_fraction = 0.5
agent_risk_limit = 0.1
tasks = [{"name": "routine", "agent_authorized": true, "risk": 0.02, "deadline": 2,
"human_authorized": false, "review_received": false}, {"name": "release", "agent_authorized": false,
"risk": 0.05, "deadline": 2, "human_authorized": true, "review_received": true}, {"name":
"sensitive", "agent_authorized": false, "risk": 0.2, "deadline": 2, "human_authorized": true,
"review_received": false}]
```

Changed case (any field not listed keeps its default value):

```
delegation_fraction = 0.9
```

Transfer case (any field not listed keeps its default value):

```
arrival_rate = 1
service_rate = 2
delegation_fraction = 1
agent_risk_limit = 0.05
tasks = [{"name": "approval", "agent_authorized": false, "risk": 0.01, "deadline": 0.5,
"human_authorized": true, "review_received": true}]
```

Questions:

1. Compute default review mean.
2. Why is changed stationary mean unavailable?
3. Does mean 1 prove the transfer receipt arrived after 0.5?

Capstone: the document-release controller

These questions accompany notebook 28. They need no inputs beyond the process described in `data/examples/capstone.json`.

1. If all preparation steps are perfect, why can terminal completion still be below one?
2. Which inputs describe the world, and which branches describe the controller's policy?
3. What would need to change before the results supported a claim about an actual agent?

Running and inspecting the original controller

A user asks the controller to release a specified document to a named recipient once its claims are supported. This constructed episode follows that request through the book's separate mechanisms. Its contract permits reading sources, preparing candidates, checking the release record, and one release submission after evidence and authorization checks. It also permits a read-only status check and a handoff to the document owner. It does not permit an unverified second submission. The budget reserves capacity for that status check and handoff before optional generation begins; no numerical utility scale from another example is imported.

The initial state contains the request, document version, recipient, current grant, source references, and remaining resources. A compact memory retains these decision facts and their provenance. It drops an

irrelevant formatting discussion, but preserves that one factual claim lacks support. The prose draft is not itself evidence that the missing claim is true.

Stage	Observation or retained state	Controller decision
Gather evidence	Source passage resolves the missing claim; version and locator retained	Continue with the same document request
Compare candidates	Draft alternatives differ in wording, with no new supporting evidence	Select the supported version; stop optional sampling
Check authority	Current grant covers the selected final document version, recipient, and release condition	Admit the specified release action
Submit release	Service times out after accepting the request for processing	Record effect as unresolved
Inspect status	Read-only status check still reports pending	Hold further submissions; prepare handoff
Transfer decision	Packet to document owner contains request identifier, evidence, and unresolved state	End autonomous action with a bounded follow-up route

Each admitted step uses Chapter 5’s normalized transition over the composite state. A failed authority or precondition check takes its explicit rejection branch, consuming any declared checking cost without executing the rejected world effect. The trace follows the admitted release branch; it does not silently discard rejected proposals.

Candidate generation changes what the controller can choose among. It does not establish that a selected candidate is supported or grant permission to send it. Here the retrieved passage supplies evidence, the selector chooses wording, and the authority check binds the selected final version and recipient before admitting a particular world-changing action. Stopping optional sampling preserves the reserved recovery capacity. A new candidate would not answer whether a release submission later took effect.

The timeout creates a different uncertainty. The controller knows it submitted a request and lacks a completed response. It does not know whether the recipient received the document. Memory therefore retains the original request identifier and the distinction between attempted, pending, and confirmed effect. The status check consumes its reserved allowance without resolving that distinction. Pending is neither verified non-application nor permission to retry; the original request may still complete.

The controller now holds further submissions and transfers the packet to the document owner through the authorized channel. The handoff names the owner, the unresolved effect, and the next day’s noon review as its deadline. If no response arrives by that deadline, the fallback keeps further submissions blocked and returns control to the requester. No human acceptance or successful delivery is assumed. The episode ends with an auditable unresolved state and a named next decision, rather than manufacturing completion from a timeout or spending another sample on uncertainty that only execution evidence can resolve.

Simulator probes

Use this small manifest to compare a matching approval with a stale approval under the same document version. If you did not extract `document-release-simulator.zip`, save the manifest as `release-probe.json` in the working folder. The `mathematical-agent-probes.zip` and `math-ai-agents-lab.zip` packages already contain the same manifest at `Companion/fixtures/release-probe.json`:

```
{
  "seed": 0,
  "runs": [
    {"version": "v2", "approval_for": "v2", "seed": 0},
    {"version": "v2", "approval_for": "v1", "seed": 0}
  ]
}
```

The two rows differ only in approval scope. Each case records its seed explicitly; the top-level seed labels the manifest.

Run the simulator with this manifest from the folder that holds scripts. With `release-probe.json` saved there, or extracted from `document-release-simulator.zip`, use the first command. With either larger package, use the second:

```
python3 scripts/document_release_simulator.py --manifest release-probe.json
python3 scripts/document_release_simulator.py --manifest Companion/fixtures/release-probe.json
```

On Windows, use `py -3` in place of `python3` if that is your Python launcher. The output is JSON: the matching row reports `authorized_release`, while the stale row reports `abstained`.

To investigate the other named faults, change one case at a time:

1. Keep `version` and `approval_for` equal to `v2`, and add `"fault": "dropped_acknowledgement"`. Expect a release proposal with `actual_effect` equal to `pending`, not `confirmed delivery`.
2. Replace that fault with `duplicate_request`. Expect `duplicate_effect`; the output records two applications rather than `authorized completion`.
3. Replace it with `late_cancellation`. Expect `recovered`; inspect the recorded `released-and-rolled-back effect` rather than assuming the original action never happened.
4. Compare the outcome, actual effect, cost, recovery time, and remaining budget across cases. You should be able to distinguish a policy proposal from an observed service effect.
5. For Chapter 10's simulator exercise, keep both fields equal to `v2` and add `"fault": "document_changed"`. Expect `abstained` with `actual_effect` equal to `restart_required`: the release option restarts instead of inheriting its predecessor's approval.
6. For Chapter 13's simulator exercise, add `"policy": "trained"` to the stale row (`approval_for` equal to `v1`), then repeat with `"policy": "fixed"`. Both report `abstained`; with `approval_for` equal to `v2`, both report `authorized_release`. The policy label does not change what the approval authorizes.

`scripts/document_release_simulator.py` makes this constructed trace reproducible. Try matching version and approval scope for `authorized release`. Then change only approval scope from `v2` to `v1`; outcome must be `abstention`. Change only fault to `dropped_acknowledgement`; proposal remains `release`, while actual effect

becomes pending and another release is blocked. Finally inject `duplicate_request` or `late_cancellation`; those are distinct effects, not proof that a request stayed safely idempotent. The manifest records seed, fault, proposal, actual effect, cost, recovery time, and remaining budget. It is a deterministic teaching harness, not a model of a production document service.

Running the original expansion probes

Extract the complete companion package and open a terminal in its root folder. Each command below reads one local fixture and prints JSON. On Windows, substitute `py -3` for `python3` when appropriate. The companion README identifies expected-output files and the chapter served by each probe.

1. Run the computer-use fixture. Inspect reached document, effect document, denial, and pending confirmation.
2. Run the learning fixture. Compare actual updated policies with the fixed baseline and separate sampled evaluation from exact expected values.
3. Run orchestration at capacity two, then capacity one. Compare elapsed time and work costs before interpreting team performance.
4. Run resources with a three-second deadline, then ten seconds. Retain zero-success procedures with an unavailable ratio.
5. Run evaluation with two repetitions. Compare task-first repetition with powers of the aggregate mean.

```
python3 scripts/computer_use_probe.py --manifest Companion/fixtures/computer-use.json
python3 scripts/agent_learning_probe.py --manifest Companion/fixtures/learning.json
python3 scripts/orchestration_probe.py --manifest Companion/fixtures/orchestration.json
python3 scripts/resource_allocation_probe.py --manifest Companion/fixtures/resources.json
python3 scripts/evaluation_probe.py --manifest Companion/fixtures/evaluation.json
```

The commands execute finite constructed programs, not language-model benchmarks. Open the local `release-console` HTML file for the visual version of the stale-coordinate example. Its effects remain inside the page.